

1. For this assignment, I used the free version of ChatGPT (GPT-5.6 Luna) that you would find on a completely new tab of the ChatGPT website, and I used Gemini's 3.6 Flash.
2. For both LLMs, I only sent one request, which consisted of the assignment requirements directly copied from the provided instructions to this assignment, prefaced with "Please follow these instructions using C:"
3. The code from each LLM will be below
Start of ChatGPT's code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_EMAILS 1000
#define SUBJECT_SIZE 500
#define CATEGORY_SIZE 30

typedef struct {
    char sender[CATEGORY_SIZE];
    char subject[SUBJECT_SIZE];
    char date[11];

    int priority;
    int dateValue;
} Email;

typedef struct {
    Email heap[MAX_EMAILS];
    int size;
} MaxHeap;

/* -----
   Convert MM-DD-YYYY into an integer that can be compared.
   YYYYMMDD gives chronological ordering.
   ----- */
int convertDate(char *date)
{
    int month, day, year;

    sscanf(date, "%d-%d-%d", &month, &day, &year);
```

```

        return year * 10000 + month * 100 + day;
    }

    /* -----
    Assign priority based on sender category.
    Higher number = higher priority.
    ----- */
int getPriority(char *sender)
{
    if (strcmp(sender, "Boss") == 0)
        return 5;
    else if (strcmp(sender, "Subordinate") == 0)
        return 4;
    else if (strcmp(sender, "Peer") == 0)
        return 3;
    else if (strcmp(sender, "ImportantPerson") == 0)
        return 2;
    else
        return 1;    /* OtherPerson */
}

    /* -----
    Determine whether email a has higher priority than email b.

    First compare sender priority.
    If equal, compare dates.
    Newer date has higher priority.
    ----- */
int higherPriority(Email a, Email b)
{
    if (a.priority > b.priority)
        return 1;

    if (a.priority < b.priority)
        return 0;

    if (a.dateValue > b.dateValue)
        return 1;

    return 0;
}

```

```

}

/* -----
   Swap two emails.
   ----- */
void swap(Email *a, Email *b)
{
    Email temp = *a;
    *a = *b;
    *b = temp;
}

/* -----
   Initialize an empty MaxHeap.
   ----- */
void initializeHeap(MaxHeap *heap)
{
    heap->size = 0;
}

/* -----
   Move an element upward until the MaxHeap property is
   restored.
   ----- */
void heapifyUp(MaxHeap *heap, int index)
{
    int parent;

    while (index > 0)
    {
        parent = (index - 1) / 2;

        if (higherPriority(heap->heap[index],
                           heap->heap[parent]))
        {
            swap(&heap->heap[index], &heap->heap[parent]);
            index = parent;
        }
        else
        {

```



```

        largest = left;
    }

    if (right < heap->size &&
        higherPriority(heap->heap[right],
                       heap->heap[largest]))
    {
        largest = right;
    }

    if (largest != index)
    {
        swap(&heap->heap[index], &heap->heap[largest]);
        index = largest;
    }
    else
    {
        break;
    }
}

}

/* -----
   Remove and return the highest-priority email.
   ----- */
Email removeMax(MaxHeap *heap)
{
    Email result;

    result.sender[0] = '\0';
    result.subject[0] = '\0';
    result.date[0] = '\0';
    result.priority = 0;
    result.dateValue = 0;

    if (heap->size == 0)
        return result;

    result = heap->heap[0];

```

```

    heap->size--;

    if (heap->size > 0)
    {
        heap->heap[0] = heap->heap[heap->size];
        heapifyDown(heap, 0);
    }

    return result;
}

/* -----
   Return the highest-priority email without removing it.
   This is needed because NEXT does not remove the email.
   ----- */
Email peek(MaxHeap *heap)
{
    Email result;

    result.sender[0] = '\0';
    result.subject[0] = '\0';
    result.date[0] = '\0';
    result.priority = 0;
    result.dateValue = 0;

    if (heap->size == 0)
        return result;

    return heap->heap[0];
}

/* -----
   Process the input file.
   ----- */
int main(int argc, char *argv[])
{
    MaxHeap heap;
    FILE *file;

    char line[1000];

```

```

char command[20];

initializeHeap(&heap);

/*
    If a filename is supplied:
        ./program test.txt

    Otherwise, input can be redirected:
        ./program < test.txt
*/
if (argc > 1)
{
    file = fopen(argv[1], "r");

    if (file == NULL)
    {
        printf("Unable to open file.\n");
        return 1;
    }
}
else
{
    file = stdin;
}

while (fgets(line, sizeof(line), file) != NULL)
{
    /* Remove newline characters. */
    line[strcspn(line, "\r\n")] = '\0';

    /* -----
        EMAIL command
        Format:
        EMAIL Category,Subject,MM-DD-YYYY
        ----- */

    if (strncmp(line, "EMAIL ", 6) == 0)
    {
        Email email;
        char *token;
    }
}

```

```

    token = strtok(line + 6, ",");

    if (token == NULL)
        continue;

    strcpy(email.sender, token);

    token = strtok(NULL, ",");

    if (token == NULL)
        continue;

    strcpy(email.subject, token);

    token = strtok(NULL, ",");

    if (token == NULL)
        continue;

    strcpy(email.date, token);

    email.priority = getPriority(email.sender);
    email.dateValue = convertDate(email.date);

    insert(&heap, email);
}

/* -----
   NEXT command

   NEXT displays the highest-priority email but
   DOES NOT remove it from the heap.
   ----- */
else if (strcmp(line, "NEXT") == 0)
{
    Email email = peek(&heap);

    if (heap.size > 0)
    {

```

```

        printf("Next email:\n");
        printf("Sender: %s\n", email.sender);
        printf("Subject: %s\n", email.subject);
        printf("Date: %s\n", email.date);
    }
}

/* -----
   READ command

   READ removes the highest-priority email without
   displaying it.
   ----- */
else if (strcmp(line, "READ") == 0)
{
    if (heap.size > 0)
    {
        removeMax(&heap);
    }
}

/* -----
   COUNT command
   ----- */
else if (strcmp(line, "COUNT") == 0)
{
    printf("There are %d emails to read.\n", heap.size);
}
}

if (file != stdin)
    fclose(file);

return 0;
}

```

End of ChatGPT's code,
Start of Gemini's Code

```

#include <stdio.h>
#include <stdlib.h>

```

```

#include <string.h>

#define MAX_LINE 256

typedef struct {
    char sender[30];
    char subject[100];
    char date[15]; // Format: MM-DD-YYYY
} Email;

typedef struct {
    Email *data;
    int size;
    int capacity;
} MaxHeap;

// Helper function to map sender category string to integer priority
// score.
// Higher number = higher priority.
int get_category_priority(const char *sender) {
    if (strcmp(sender, "Boss") == 0) return 5;
    if (strcmp(sender, "Subordinate") == 0) return 4;
    if (strcmp(sender, "Peer") == 0) return 3;
    if (strcmp(sender, "ImportantPerson") == 0) return 2;
    return 1; // OtherPerson
}

// Converts MM-DD-YYYY string into YYYYMMDD integer for chronological
// comparison
int date_to_int(const char *date) {
    int m, d, y;
    sscanf(date, "%d-%d-%d", &m, &d, &y);
    return y * 10000 + m * 100 + d;
}

// Returns > 0 if email A has higher priority than email B, < 0 if B > A,
// 0 if equal
int compare_emails(const Email *a, const Email *b) {
    int prioA = get_category_priority(a->sender);
    int prioB = get_category_priority(b->sender);

```

```

    if (prioA != prioB) {
        return prioA - prioB;
    }
    // Same category: newer date wins
    return date_to_int(a->date) - date_to_int(b->date);
}

// Initialize the Heap
MaxHeap* create_heap(int initial_capacity) {
    MaxHeap *heap = (MaxHeap*)malloc(sizeof(MaxHeap));
    heap->capacity = initial_capacity;
    heap->size = 0;
    heap->data = (Email*)malloc(sizeof(Email) * heap->capacity);
    return heap;
}

void swap(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify up when inserting
void heapify_up(MaxHeap *heap, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (compare_emails(&heap->data[index], &heap->data[parent]) > 0) {
            swap(&heap->data[index], &heap->data[parent]);
            index = parent;
        } else {
            break;
        }
    }
}

// Heapify down when extracting max
void heapify_down(MaxHeap *heap, int index) {
    int largest = index;
    int left = 2 * index + 1;

```

```

    int right = 2 * index + 2;

    if (left < heap->size && compare_emails(&heap->data[left],
&heap->data[largest]) > 0) {
        largest = left;
    }
    if (right < heap->size && compare_emails(&heap->data[right],
&heap->data[largest]) > 0) {
        largest = right;
    }
    if (largest != index) {
        swap(&heap->data[index], &heap->data[largest]);
        heapify_down(heap, largest);
    }
}

void insert_email(MaxHeap *heap, Email e) {
    if (heap->size == heap->capacity) {
        heap->capacity *= 2;
        heap->data = (Email*)realloc(heap->data, sizeof(Email) *
heap->capacity);
    }
    heap->data[heap->size] = e;
    heapify_up(heap, heap->size);
    heap->size++;
}

void read_email(MaxHeap *heap) {
    if (heap->size == 0) return;

    heap->data[0] = heap->data[heap->size - 1];
    heap->size--;
    if (heap->size > 0) {
        heapify_down(heap, 0);
    }
}

void free_heap(MaxHeap *heap) {
    free(heap->data);
    free(heap);
}

```

```

}

int main(void) {
    MaxHeap *heap = create_heap(10);
    char line[MAX_LINE];

    while (fgets(line, sizeof(line), stdin) != NULL) {
        // Strip trailing newline character
        line[strcspn(line, "\r\n")] = 0;
        if (strlen(line) == 0) continue;

        if (strncmp(line, "EMAIL ", 6) == 0) {
            Email e;
            char *token_str = line + 6;

            // Parse comma-delimited fields
            sscanf(token_str, "%29[^,],%99[^,],%14s", e.sender, e.subject,
e.date);

            // Trim leading/trailing whitespace if present
            char *s = e.sender;
            while(*s == ' ') s++;

            insert_email(heap, e);
        }
        else if (strcmp(line, "COUNT") == 0) {
            printf("There are %d emails to read.\n", heap->size);
        }
        else if (strcmp(line, "NEXT") == 0) {
            if (heap->size > 0) {
                printf("Next email:\n");
                printf("Sender: %s\n", heap->data[0].sender);
                printf("Subject: %s\n", heap->data[0].subject);
                printf("Date: %s\n", heap->data[0].date);
            }
        }
        else if (strcmp(line, "READ") == 0) {
            if (heap->size > 0) {
                read_email(heap);
            }
        }
    }
}

```

```
    }  
}  
  
    free_heap(heap);  
    return 0;  
}  
End of Gemini's code  
0. Comparing
```

Correctness

Execution Time

Space Usage

Both programs allocate a fixed amount of memory upfront using defined constants. ChatGPT sets a hard limit with `MAX_EMAILS` (1000) and `SUBJECT_SIZE` (500), which reserves memory right away whether you store 5 emails or 500. While this static memory usage is small enough that it won't slow down a computer, it does waste unused space for short emails.

Maintainability

While both LLMs generated different types of defined constants, Gemini had only 1, while ChatGPT had three. Gemini defined a constant `MAX_LINE` of 256, and ChatGPT defined constants `MAX_EMAILS` of 1000, `SUBJECT_SIZE` of 500, and `CATEGORY_SIZE` of 30. While ChatGPT's constants seem more appealing due to being more defined, the way they're implemented actually breaks when you test it.

For documentation, ChatGPT's code has multi-line comments to explain how each function works, while Gemini has in-line comments from time to time, sometimes to explain functions, but also to describe what happens with specific lines. I personally prefer ChatGPT's multi-line commenting, but I think it should be combined with the way Gemini uses in-line comments to make the code as easy to follow as possible.

4. Comparing

- a. Correctness: When tested against edge cases, both models had correctness issues out of the box. ChatGPT's program runs into a heap sorting bug caused by its priority comparison logic.

When multiple emails are removed using READ commands, the priority function fails to handle date ties and orderings strictly, which causes heapifyDown to stop early and leaves the wrong email at the top of the queue. On the other hand, Gemini handles heap deletions cleanly, but its date parsing logic breaks on single-digit dates (like 1-1-2026 versus 01-01-2026), causing it to miscalculate chronological order. Neither program produced completely correct results without manual debugging.

- b. Execution Time: Both models use a binary MaxHeap structure to manage the email queue, meaning operations like insertion and deletion take logarithmic time ($O(\log N)$). Because of this efficient structure, execution time for both programs is virtually instant in testing. In practice, performance is almost entirely limited by reading lines from the input file (fgets) rather than processing memory.
 - c. Space Usage: Both models allocate a fixed amount of memory upfront using pre-defined constants. ChatGPT sets a hard limit with MAX_EMAILS (1000) and SUBJECT_SIZE (500), which reserves memory on the stack immediately whether the queue holds 5 emails or 500. While this static memory allocation requires only a fraction of a megabyte and won't impact performance on modern systems, it does waste unused memory when processing shorter emails.
 - d. Maintainability: While both LLMs generated different types of defined constants, Gemini included only one, whereas ChatGPT defined three. ChatGPT defined constants for MAX_EMAILS (1000), SUBJECT_SIZE (500), and CATEGORY_SIZE (30), which makes the code easier to configure if buffer sizes need to change later. For documentation, ChatGPT uses multi-line comments before each function to explain what it does, while Gemini uses in-line comments throughout the code to explain specific lines. Combining ChatGPT's high-level function overviews with Gemini's detailed inline comments creates the most maintainable code.
5. I decided to move forward with the code that Gemini generated, because the bugs present when I examined them were less problematic in my mind. However, the methodology of ChatGPT,

namely the extensive commenting, and use of whitespace is definitely more aligned with how I like to organize my code, which is something I will make Gemini's code more alike to when I improve it.

6. Changes go here,

- a. Improved input line & struct buffer management. I increased the MAX_LINE constant due to encountering string truncation on long lines. I also adjusted various Email struct field sizes. Overall, this change prevents buffer overflows and truncation issues, allowing the program to safely process long inputs.
- b. Flexible date parsing for non-zero-padded dates. During additional edge-case testing, I noticed that dates without leading zeros caused sorting issues, so I updated the date_to_int() function to accommodate flexible date formats.
- c. In-place whitespace trimming. Updating a local character pointer (char *s) initially bypassed leading spaces without actually modifying e.sender. This caused category priority assignment issues. Now, if leading spaces are detected, it uses memmove to shift the string in place.